

FoodBridge — React Frontend Mentorship

Day 7 Notes — Revision Day: Routing, Context API, Interview Prep

No new features built today — this was a dedicated revision session, going deep on concepts already implemented in the project (routing internals, `useNavigate` vs `Navigate`, Context API), converted into interview-ready answers.

1. BrowserRouter vs Routes vs Route

- `BrowserRouter` — enables URL-based navigation app-wide using the browser's History API; lets the app change the URL without a full page reload. Everything else (`Route`, `Link`, `useNavigate`, `Navigate`) depends on being nested inside it.
- `Routes` — a container that looks at the current URL and picks exactly ONE matching `Route` (out of all its children) to render.
- `Route` — one single mapping: 'this specific path -> this specific component.'

Correction made during revision: `Routes` (plural) is the container/parent; `Route` (singular) is the individual entry inside it — not the other way around.

2. Why No Full Page Reload (SPA Behavior)

Traditional links (`<a href>`) discard the whole page and request a brand new HTML file from the server on every click — causing a visible flash/reload. React Router's `Link` instead just updates the URL in the address bar (via `BrowserRouter`) and swaps only the matching component via `Routes`, while persistent UI like the `Navbar` stays untouched. This is why React apps are Single Page Applications — only ONE real HTML file (`index.html`) is ever loaded from the server.

3. `useNavigate()` vs `<Navigate />`

```
useNavigate()  -> hook, returns a FUNCTION (navigate)
                  called imperatively, works anywhere in code:
                  event handlers, .then(), useEffect

<Navigate />   -> a COMPONENT, only works if returned as JSX
                  during a component's render (declarative)
```

Where each is used in FoodBridge:

```
// Login.jsx – useNavigate, inside .then() after API call succeeds
const navigate = useNavigate()
axiosInstance.post("/users/login", credentials).then((res) => {
  navigate(`/${decoded.role.toLowerCase()}/dashboard`)
})

// ProtectedRoute.jsx – <Navigate />, returned directly during render
if (!user) {
  return <Navigate to="/login" />
}
```

Why they can't be swapped:

is JSX — it only has effect if it's part of a component's render output. Returning it from inside `.then()` does nothing, because `.then()` is a plain callback that runs long after rendering already finished — nothing is 'listening' for JSX there. `navigate()` is just a function call — it works anywhere, anytime, including inside async callbacks, because it doesn't depend on being part of a render output.

Deeper mechanism: `navigate()` indirectly causes a re-render because it changes React Router's own internal state (same underlying mechanism as any `setState`, just managed by the library). achieves the same underlying re-render, but only once React actually renders it as JSX — the difference is HOW each one gets triggered (called vs rendered), not what they ultimately do.

Extra `navigate()` options (to implement later, not yet used in the project):

- `navigate(path, { replace: true })` — replaces current history entry instead of adding a new one; planned for Login's post-login redirect so the back button doesn't return to the login form
- `navigate(-1)` — goes back one step in history, like the browser back button; potential future use for a form Cancel button
- `navigate(path, { state: {...} })` — passes data to the next page without putting it in the URL; not needed yet

4. Interview-Ready Answer: Routing Basics

Q: How does client-side routing work in React, and what's the difference between `BrowserRouter` and `Routes`?

A: Traditional multi-page sites reload the whole page on every navigation — discard current page, request new HTML, rebuild from scratch. React Router avoids this with client-side routing. `BrowserRouter` uses the browser's History API to sync the app with the current URL and update it without a real page reload; everything else (`Route`, `Link`, `useNavigate`, `Navigate`) depends on it being present higher in the tree. `Routes` performs the actual matching — it looks at the current URL and renders exactly one matching `Route`, swapping only that piece of UI while persistent elements like a `Navbar` stay the same. This is why React apps are called SPAs — only one HTML file loads; the rest is just component swapping synced to the URL.

Q: Explain `useNavigate` and `<Navigate />` — how they work and what else they can do?

A: Both trigger navigation but differ in how they're triggered. `useNavigate` returns a function called imperatively — e.g. after an async API call resolves inside `.then()`. Calling `navigate()` updates React Router's internal URL state, which causes a re-render of the matched route, the same underlying mechanism as any `setState` call, just managed internally by the library. It also accepts options like `{ replace: true }` to avoid adding a new history entry, or `navigate(-1)` to go back. `<Navigate />` is a component — JSX only has effect when actually rendered, so it's used where the redirect decision IS the render output itself, like in `ProtectedRoute` checking auth state. The rule of thumb: use `navigate()` for redirects reacting to logic/events, use `<Navigate />` for redirects that are a render-time decision.

5. Interview-Ready Answer: Context API

Q: Explain the Context API — why it's needed, and how you implemented it in a real project.

A: Context API solves prop drilling — passing data through many layers of components that don't need it, just to reach one that does. In FoodBridge, the logged-in user's JWT/role/email needed to be accessible in `Navbar`, `ProtectedRoute`, and eventually API calls, so Context API was used instead of manual prop passing.

```

export const AuthContext = createContext()

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null)
  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  )
}

// main.jsx – activate app-wide
<AuthProvider><App /></AuthProvider>

// any component – read from context
const { user, setUser } = useContext(AuthContext)

```

Real usage across the app: Login calls `setUser({token, role, email})` after successful login; `AuthContext`'s own `useEffect` restores user from `localStorage` on app startup; `Navbar` reads user to show correct links and calls `setUser(null)` on logout; `ProtectedRoute` reads user to verify auth + role before rendering a protected page.

6. Interview-Ready Answer: State Updates Are Asynchronous

Q: Is `setState` synchronous or asynchronous in React? What happens if you read state right after calling its setter?

A: State updates in React are asynchronous/batched, not applied instantly. Calling a setter like `setUser(...)` schedules an update; the new value is only available after the component re-renders, not on the very next line. Reading the state variable immediately after its setter still returns the OLD value. In FoodBridge, after calling `setUser(...)` with freshly decoded JWT data, trying to immediately read `user.role` on the next line still returned null. Fix: use the original source data (`decoded.role`) directly instead of relying on the state variable right after setting it.

Q (follow-up): Why does React do this instead of updating instantly?

A: Performance — batching multiple state updates together before a single re-render avoids redundant, wasteful re-renders that would happen if every `setState` call triggered an immediate synchronous update.

Q (follow-up): How do you get the updated value immediately?

A: You can't synchronously on the next line — that's inherent to how state works. Use the original source value already on hand instead of the state variable, or use `useEffect` with that state as a dependency to react after the update has actually applied.

7. Recalled Mistake: Rules of Hooks Violation

From Day 4: `useContext(AuthContext)` was originally called OUTSIDE the `Login` component, at the top level of the file, before function `Login()` even started.

```

// WRONG – outside any component
const { user, setUser } = useContext(AuthContext)
function Login() { ... }

// CORRECT – inside the component function
function Login() {
  const { user, setUser } = useContext(AuthContext)
  ...
}

```

Rule: hooks (useState, useContext, useEffect, etc.) can only be called inside a component function or a custom hook — never at file top-level, inside loops, or inside conditionals. React needs to track hooks in a consistent order per render to know which state/context belongs to which call; calling a hook outside a component gives React no render cycle to attach it to, causing an 'Invalid hook call' error.

Full Rules of Hooks interview answer + more depth to be covered next session (Day 8).

8. Next Steps (Day 8)

- Full Rules of Hooks — interview-ready explanation
- Connect Register form to POST /users endpoint (build data object, handle organizationName -> null for non-NGO, ternary operator)
- Attach JWT automatically to protected requests via Axios interceptor
- Add auto-logout when token expires (using exp field from decoded JWT)
- Implement navigate(path, { replace: true }) in Login's post-login redirect